

文件系统崩溃一致性

计算机系统在运行时可能会因遇到各种意外情况而重启，例如计算机意外掉电，或者软件缺陷导致系统崩溃。重启后，计算机会将大多数硬件资源重新初始化，以一种“干净”的状态开始工作——除了存储设备。应用程序通常会预期保存在存储设备中的数据即使经历了意外掉电或崩溃，重启后依然有效。对于应用程序和存储设备的中间层——文件系统来说，这个预期变成：无论在什么时候发生哪种软硬件错误，文件系统应始终能持久且完好地保存应用程序的文件。

然而，面对任何时刻均可能发生的意外情况，文件系统想要提供这个保障并非一件容易的事情。在应用程序看来，它们只需要简单地调用 `open`、`write`、`fsync` 等接口即可完成数据的持久化保存，这些接口只有执行和未执行两种状态。实际上在每个接口背后，文件系统需要执行大量工作，如空间分配、数据拷贝、元数据更改和数据结构修改等，文件系统在完成一个接口的功能时往往涉及存储设备上多处数据的修改。若在执行这些修改的过程中发生掉电或崩溃等情况，那么一个文件操作可能只做到一半，即只有一部分修改被写入存储设备。这有可能导致存储设备所保存的数据（包括文件数据和元数据）之间的一些内在关系（即一致性）遭到破坏。这种问题通常被称为崩溃一致性（Crash Consistency）问题。

本章将首先以创建文件为例对文件系统操作中的崩溃一致性进行直观的介绍；然后介绍多种保证文件系统崩溃一致性的方法，包括在掉电或崩溃发生后对文件系统一致性的检查和修复，使用日志和写时拷贝等多修改原子更新技术保证文件操作的原子完成，使用 Soft updates 技术严格维护持久化顺序以保持存储设备上元数据的一致性；最后以日志结构文件系统（Log-structured File System, LFS）为例，介绍如何使用日志结构配合写时拷贝技术对整个文件系统的数据和元数据进行管理。

12.1 崩溃一致性

本节主要知识点

❑ 在创建文件的过程中可能会遇到哪些崩溃一致性问题？

我们以第 11 章中介绍的基于 inode 的文件系统为例，介绍在创建一个新的文件时涉及的文件系统崩溃一致性。创建一个新的常规文件的总流程大致可以分为三步^①。

- 1. 分配 inode。新的常规文件需要使用一个新的 inode 结构进行保存。因此在创建文件时需先为新的文件分配新的 inode 结构。这一步骤需要在 inode 分配器的位图中查找空闲的 inode，并将其对应的位标记为 1，表示该 inode 已被使用。
- 2. 初始化 inode。在分配并得到 inode 之后，文件系统需要对该 inode 进行初始化操作，即将新文件的信息保存在 inode 结构之中。
- 3. 增加目录项。最后，需要在父目录中添加新的目录项，保存新文件的文件名和 inode 号。

图 12.1 给出了创建文件过程中所需要修改的三处结构。可以看出，一般情况下三处结构处于存储设备的不同位置，文件系统需要发出三个对存储设备的写请求才能完成对这些结构的修改。如果在此过程中发生了掉电或崩溃等情况，存储设备上保存的数据可能会存在以下几种情况。

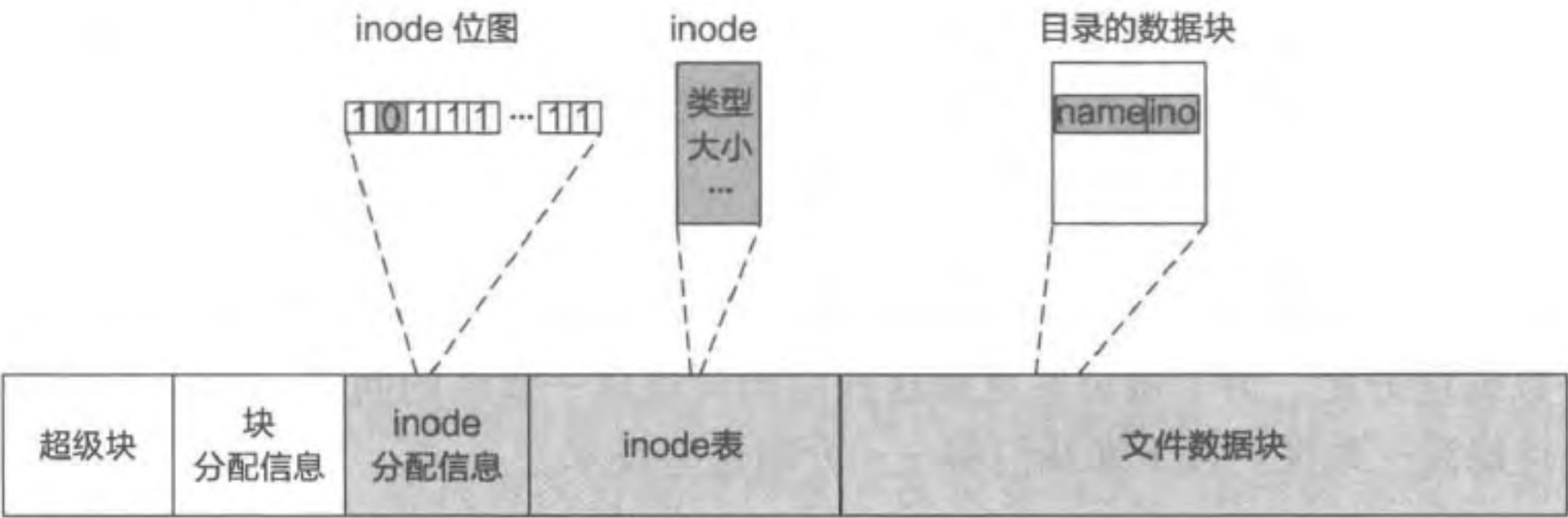


图 12.1 简化的文件创建过程中涉及的结构

情况 1：只有分配 inode 的操作保存在设备中。在这种情况下，由于增加目录项的操作并没有保存在存储设备中，新创建的文件无法在文件系统中被观测到。但是由于对应的 inode 在分配信息中已经被标记为占用，而实际上该 inode 并未被任何文件所使用，该 inode 将无法被释放，造成 inode 空间的泄漏。随着越来越多的 inode 空间被泄漏，文件系统中所能保存的文件将越来越少。这种情况实际上违反了 inode 分配信息与 inode 结构

① 在实际情况中，文件创建还需要更多的操作。为了讨论的简洁性，本章对该过程进行了简化。

的实际使用之间的一致性。

情况 2：只有初始化 inode 的操作保存在设备中。同样，由于增加目录项的操作并没有保存在存储设备中，新创建的文件无法在文件系统中被观测到。同时，由于 inode 的分配信息未被修改，后续的创建文件操作依然可以使用该 inode 结构，因此并未产生空间泄漏。这种情况的出现并不会造成文件系统的一致性问题。

情况 3：只有增加目录项的操作保存在设备中。由于增加目录项的操作被持久化在存储设备中，当文件系统遍历该目录时，能够看到新文件对应的文件名和 inode 号。然而由于该 inode 结构中的数据未被初始化，文件系统尝试访问该 inode 时，会访问到未初始化的数据，从而产生错误。同时，由于该 inode 号对应的分配信息未被持久化，在后续的文件操作中，该 inode 可能会被再次分配给其他文件。这将导致该 inode 错误地被两个不同的文件使用，造成数据丢失、泄露和被篡改等问题。这种情况违反了 inode 分配信息、inode 结构与目录项中所保存的 inode 号之间的一致性，即所有目录项中所保存的 inode 号皆应被分配且初始化。

情况 4：只有分配 inode 的操作未保存在设备中。该情况下，由于分配 inode 的信息未被持久化，会造成情况 3 中的目录项错误地指向未分配 inode 结构的情况，从而造成正确性和安全性等问题。

情况 5：只有初始化 inode 的操作未保存在设备中。与情况 3 相似，如果只有 inode 的初始化操作未被持久化，则文件系统在后访问该文件时，会访问到未初始化的数据，从而产生错误。

情况 6：只有增加目录项的操作未保存在设备中。与情况 1 类似，当增加目录项的操作未被持久化时，会造成 inode 资源泄露的问题。

表 12.1 中给出了上述六种情况的汇总。在以上的介绍中，我们使用一个简化后的文件创建过程作为示例。实际的文件创建过程一般还涉及多种其他结构中的数据，如父目录文件的访问和修改时间，系统占用的 inode 数量统计，以及在增加目录项时需要进行的额外数据块分配。为了避免在更新这些结构时造成一致性的问题，文件系统采用多种方法保持崩溃一致性。接下来我们将一一介绍这些技术。

表 12.1 崩溃发生时的多种情况

崩溃发生时 的情况	文件系统结构			是否影响使用	典型问题
	inode 位图	inode 结构	目录项		
情况 1	持久	未持久	未持久	否	资源泄露
情况 2	未持久	持久	未持久	否	无
情况 3	未持久	未持久	持久	是	信息错乱
情况 4	未持久	持久	持久	是	信息错乱
情况 5	持久	未持久	持久	是	信息错乱
情况 6	持久	持久	未持久	否	资源泄露

12.2 同步写入与文件系统一致性检查

本节主要知识点

- 文件系统的同步写入有什么问题？
- 文件系统如何检查和修复其保存的数据与元数据的不一致？

12.2.1 同步写入

对于一个文件系统操作，文件系统会按照一定的顺序执行其中的各个步骤。如在创建文件的过程中，文件系统一般会先分配 inode，再进行初始化，最后在父目录中增加目录项。为了尽量避免文件系统在崩溃后产生一致性问题，文件系统可以使用同步写入保证修改按序完成。换句话说，文件系统每执行完一个步骤，就需要马上将其对应的修改写入存储设备中。在确认修改已经持久化在存储设备中之后，文件系统再执行下一步修改。这样虽然不能保证一个操作中的多个步骤的原子完成，但至少保证了各个修改按照既定顺序进行持久化。通过合理安排一个文件操作中各个步骤的顺序，可以避免出现指向未初始化数据等比较严重的不一致情况^①。

然而，如果严格按照顺序进行操作，文件系统的每步操作都需要访问存储设备。存储设备的访问速度一般要远小于处理器计算的速度和访问内存的速度。例如，访问一次固态硬盘需要十几到几十微秒，访问机械磁盘需要若干毫秒，而访问一次内存只需要 100 纳秒左右。如果文件系统每步操作都访问存储设备，文件系统的性能将受限于存储设备的访问速度。为了避免访问存储设备成为性能瓶颈，一些文件系统并不保证对所有数据结构上的写入都严格按照顺序进行。通过将向存储设备的写入请求推迟、重新排序、合并等方法，可以减少对存储设备的访问，提高文件系统性能。举例来说，在为文件分配数据空间时，需要分配数据块。如果严格按照同步写入的方式，在分配结果返回前，文件系统需要将分配信息的更改写入存储设备中。因此，当连续分配两个块得到 X 和 $X+1$ 时，会在每次分配时产生一次设备写入。由于表示块 X 和块 $X+1$ 的位大概率在同一个存储块中，如果推迟对存储设备的写入，则这两个分配引发的写入请求可以被合并成一个，从而减少写入请求以提升性能。虽然这种方式可以提升文件系统的性能，但在发生掉电或崩溃等事件时，存储设备中保存的信息有更大的概率出现不一致的情况。例如，在分配数据块的例子中，如果发生崩溃，存储设备中保存的数据可能显示数据块已被某个文件使用，但是分配信息中显示该数据块是未分配状态。

^① 在 12.5 节介绍 Soft updates 时会有更加详细的例子。